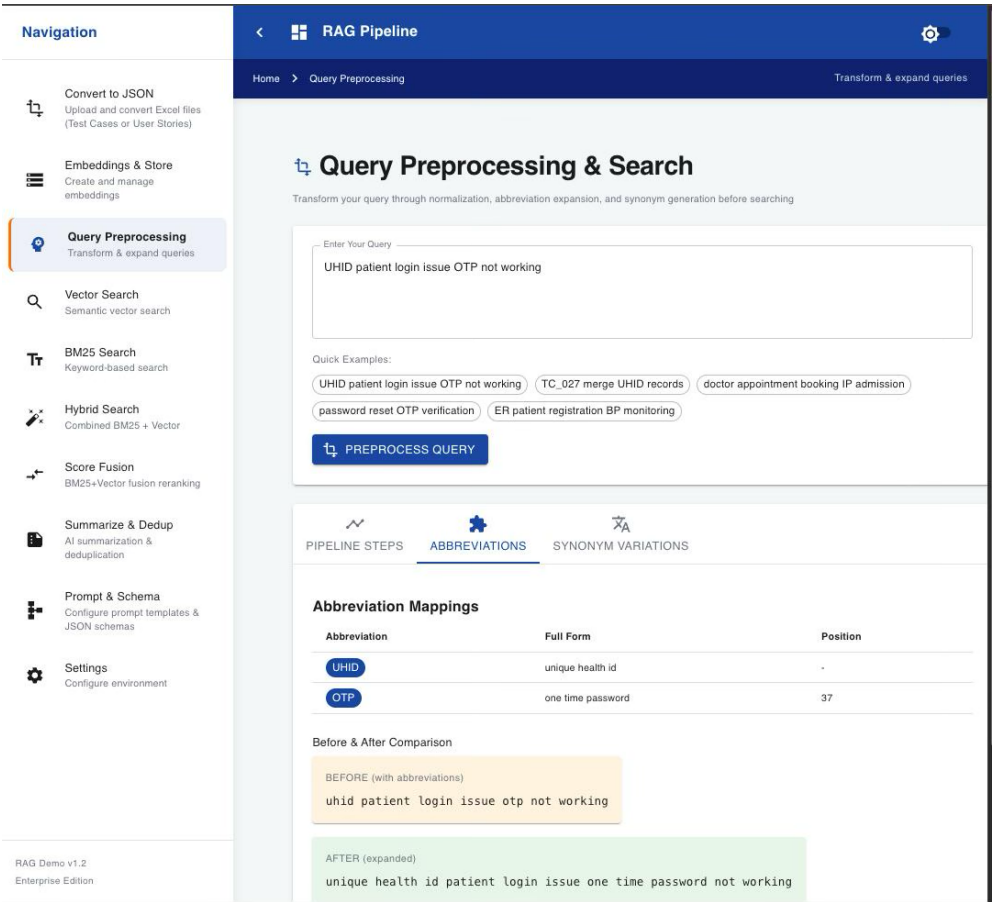
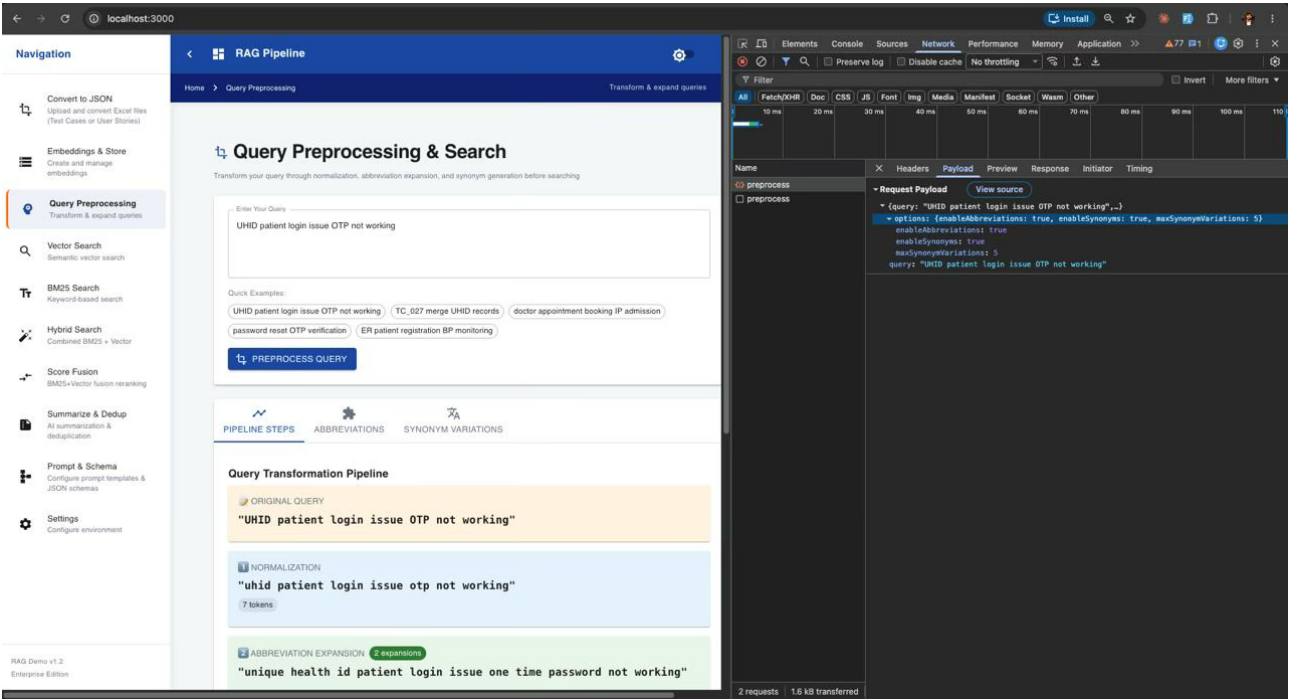
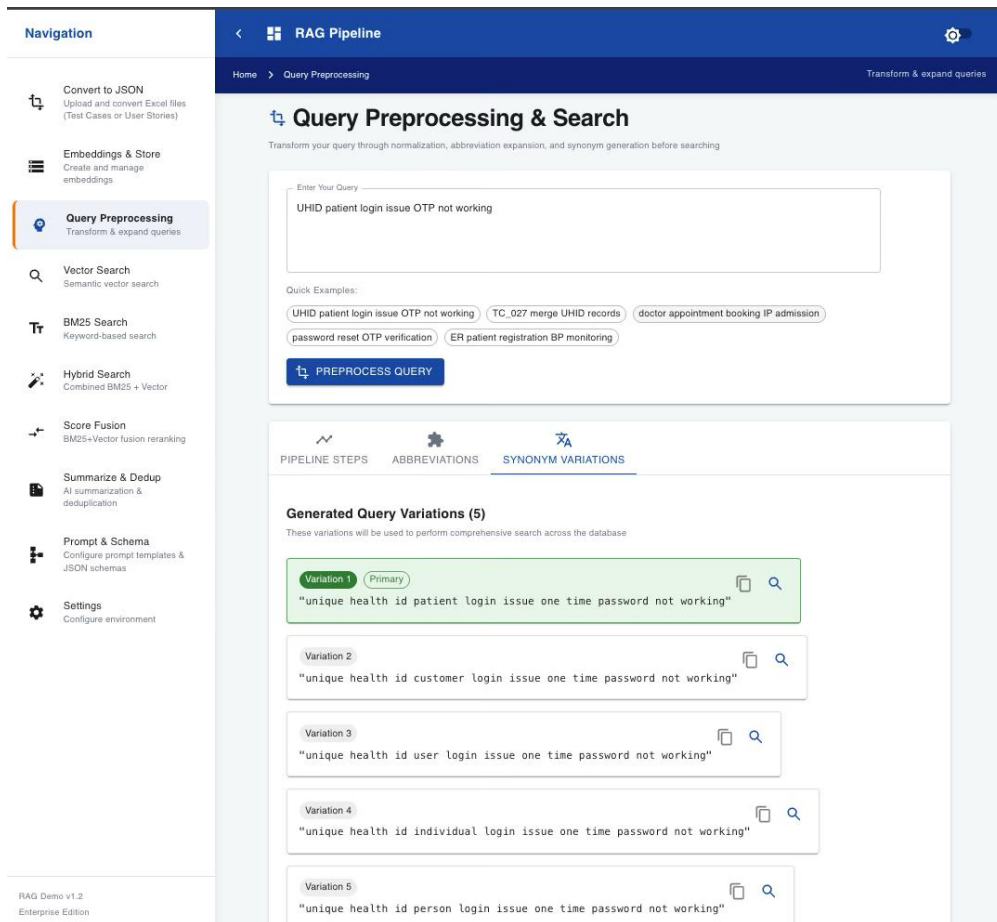


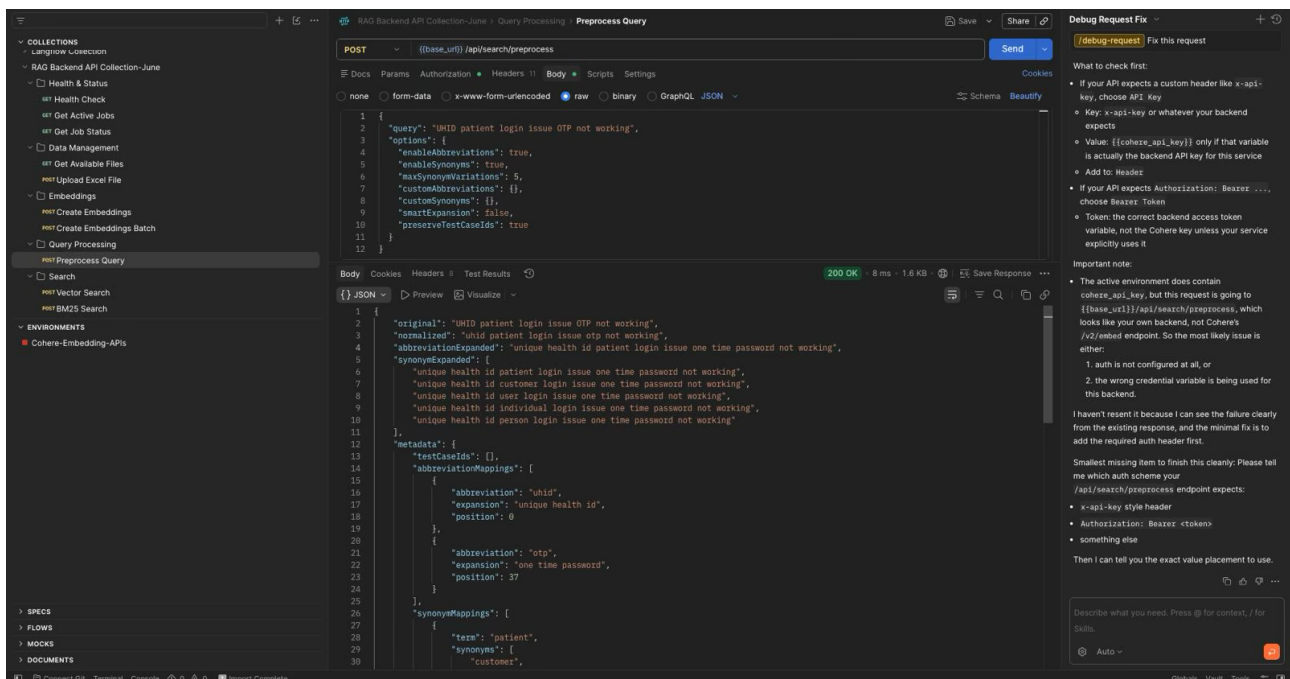
ASSIGNMENT 1: EXPLORATION OF DIFFERENT COMPONENTS OF RAG APPLICATION AND API EXECUTION IN POSTMAN

Step 1: Query Preprocess executed in RAG UI. The preprocess api executed in the backend to process the request. Normalization, Abbreviation and Synonym creation was performed





Pre-processing API executed from Postman collection returns the same results as the RAG UI



Step 2: Vector search for the entered query returns 5 probable best results

The screenshot displays the RAG Pipeline interface on the left and a browser window on the right. The RAG Pipeline interface shows a search query "Share Diagnostic Reports with Patients via WhatsApp" and its results. The browser window shows the search results in a table.

Search Query: "Share Diagnostic Reports with Patients via WhatsApp"

Search Results:

Test Case	Similarity	Description	Source
Consultant Dashboard - My Ap	95.3%	Verify user able to Save Diagnosis	Doctors-Wards
Reports - Clinical Reports - Vix	95.2%	Verify user able to view Clinical Report	Doctors-Wards
Reports - Clinical Reports - Vix	95.1%	Verify user able to view Clinical Report	Doctors-Wards
Doctor Dashboard - OP Dashb	95.1%	Verify user able to Save Diagnosis	Doctors-Wards

Browser Window:

2 requests, 5.3 kB transferred

Executing the vector search API in postman generates similar results

The screenshot displays the Postman interface showing a POST request to the RAG Backend API Collection-June. The request body is a JSON object, and the response is a JSON object.

Request:

```
{
  "query": "Share Diagnostic Reports with Patients via WhatsApp",
  "limit": 5,
  "filters": {
    "module": "",
    "priority": "",
    "risk": "",
    "automationManual": ""
  }
}
```

Response:

```
{
  "success": true,
  "query": "Share Diagnostic Reports with Patients via WhatsApp",
  "filters": {
    "module": "",
    "priority": "",
    "risk": "",
    "automationManual": ""
  },
  "results": [
    {
      "_id": "6abb39a32035f117477945c5",
      "module": "Doctors-Wards",
      "id": "TC_5263",
      "preRequisites": "1. Patient Checkedin",
      "title": "Consultant Dashboard - My Appointments - OP Dashboard - Consultations - Diagnosis",
      "description": "Verify user able to Save Diagnosis",
      "steps": "1. Launch the application\r\n2. Login to the application with valid credentials.\r\n3. Click on Patient Service Tab and select Doctors and wards from the drop down.\r\n4. Navigate to Functions->Transactions->Consultant Dashboard\r\n5. Click on Patients name under My Appointments Section\r\n6. Click on Diagnosis Tab\r\n7. Enter Disease Name and Click on Search\r\n8. Select disease and click on Add\r\n9. Click on Save as Draft",
      "expectedResults": "Draft Saved Successfully Message should be displayed",
      "automationManual": "Automated",
      "priority": "P2"
    }
  ]
}
```

Step 3: Perform BM25 search in RAG UI

The screenshot displays the RAG Pipeline interface for a BM25 Keyword Search. The search query is "Generate test cases for GDPR" with a limit of 10. The results show two items: "TC_5781" and "TC_228", both with scores around 13.3. The interface includes a navigation sidebar, a search bar, and a results list. A browser window on the right shows the network request details for the search.

BM25 API call from Postman retrieves same results

The screenshot shows a Postman API client with a POST request to the endpoint `/(base_url)/api/search/bm25`. The request body is a JSON object with the following structure:

```
{
  "query": "Generate test cases for GDPR",
  "limit": 10,
  "filters": {
    "module": "",
    "priority": "",
    "risk": ""
  },
  "fields": [
    "id",
    "title",
    "description",
    "steps",
    "expectedResults",
    "module"
  ]
}
```

The response is a JSON array of search results, including "TC_5781" and "TC_228".

Step 4: Hybrid Search performed with 30% weightage to BM25 and 70% weightage to vector search

The screenshot displays the RAG Pipeline interface on the left and a browser window on the right. The RAG Pipeline interface shows the 'Hybrid Search (BM25 + Vector)' section. The search query is 'Share Diagnostic Reports with Patients via WhatsApp'. The search weights are set to BM25 Weight: 30% and Vector Weight: 70%. The search results show 10 results, with the top result being 'TC_1174' with a hybrid score of 80.8%.

The browser window shows the search results for the query 'Share Diagnostic Reports with Patients via WhatsApp'. The results are displayed in a table with columns for Name, Headers, Payload, Preview, Response, Initiator, and Timing. The first result is 'hybrid' with a request payload of '{query: "Share Diagnostic Reports with Patients via WhatsApp", limit: 10, filters: {}, bm25Weight: 0.3, vectorWeight: 0.7}' and a response of 'Found 10 test cases in 1302ms. BM25: 161ms, Vector: 1138ms. Cost: \$0.000001 (14 tokens)'.

Hybrid search executed from postman API with same weightage and user query returns same output

The screenshot shows a Postman interface with a collection named 'RAG Backend API Collection-June'. The selected API endpoint is 'POST /api/search/hybrid'. The request body is a JSON object with the following structure:

```
{
  "query": "Share Diagnostic Reports with Patients via WhatsApp",
  "limit": 10,
  "filters": {
    "module": "",
    "priority": "",
    "risk": ""
  },
  "bm25Weight": 0.3,
  "vectorWeight": 0.7,
  "bm25Fields": [
    "id",
    "title",
    "description",
    "steps",
    "expectedResults",
    "module"
  ]
}
```

The response is a JSON object with the following structure:

```
{
  "vector": 0.7,
  "results": [
    {
      "_id": "6a6b39e12035f11747794c9f",
      "module": "IP digital",
      "id": "TC_1174",
      "title": "Investigation report of patients",
      "description": "Investigation report of patients",
      "steps": "1. Launch the application\r\n2. Login to the application with valid credentials.\r\n3. Landing screen - should be able to view multiple tabs like My patients, all patients, new patients, discharge plan, tasklist, drug administration, drug inventory, handover takeover & patient check out\r\n5. Select on investigation orders option of patient label\r\n6. Click on lab report\r\n7. Lab reports will be generated\r\n8. Expected Results: To view lab reports of the patients",
      "expectedResults": "To view lab reports of the patients",
      "automationManual": "Automated",
      "priority": "P2",
      "risk": "Low",
      "createdAt": "2024-07-30T11:47:45.536Z",
      "bm25Score": 10.40334701538086,
      "bm25ScoreNormalized": 0.5853554201571279,
      "vectorScore": 0.9009127616882324,
      "vectorScoreNormalized": 0.9009127616882324,
      "hybridScore": 0.8062455592289811
    }
  ]
}
```

Step 5: Score reranking using GROQ AI

Navigation

Convert to JSON

Embeddings & Store

Query Preprocessing

Vector Search

BM25 Search

Hybrid Search

Score Fusion

Summarize & Dedup

Prompt & Schema

Settings

RAG Pipeline

Home > Score Fusion

BM25+Vector fusion reranking

Search Query: Share Diagnostic Reports with Patients via WhatsApp

Top K: 50, Limit: 10

SEARCH

Groq AI Reranking Results: Found 10 test cases ranked by Groq AI

Timing: Total: 3144ms, Method: BM25 + Vector → Groq AI Semantic Reranking

AFTER RERANKING (FINAL) BEFORE RERANKING (ORIGINAL) COMPARISON TABLE

After Groq AI Reranking (10 results)

TC_2736, Rank Score: 0.8906, Was #4

AHCSummary -> AHCSummary -> Diagnostic & Imaging report

Module: AHC, Risk: Low

Description: Verify Diagnostic & Imaging report

Ranking Scores: BM25 Score: 16.2626, Groq Rerank: 0.8906

TC_1174, Rank Score: 0.8200, Was #8

Investigation report of patients

Module: IP digital, Risk: Low

Description: Investigation report of patients

Ranking Scores: BM25 Score: 10.4033, Groq Rerank: 0.8200

TC_4552, Rank Score: 0.7800, Was #62

Wards Dashboard - Radiology Report

Module: Wards-Doctors, Risk: Low

Elements

Console

Sources

Network

Performance

Memory

Application

Filter

Fetch/XHR

Doc

CSS

JS

Font

Img

Media

Manifest

Socket

Wasm

Other

50,000 ms

100,000 ms

150,000 ms

200,000 ms

250,000 ms

300,000 ms

350,000 ms

400,000 ms

Name

hybrid

distinct

rerank

Request Payload

View source

query: "Share Diagnostic Reports with Patients via WhatsApp", limit: 10, rerankTopK: 50, filters: {}, limit: 10

query: "Share Diagnostic Reports with Patients via WhatsApp"

rerankTopK: 50

useGroqOnly: true

Step 6: Deduplication to remove duplicates

Navigation

Convert to JSON

Embeddings & Store

Query Preprocessing

Vector Search

BM25 Search

Hybrid Search

Score Fusion

Summarize & Dedup

Prompt & Schema

Settings

RAG Pipeline

Home > Summarize & Dedup

AI summarization & deduplication

Search Summarization & Deduplication

Search, remove duplicates, and generate AI-powered summaries of test cases

Search Query: merge UHID records

Search Type: Vector, Limit: 20

Quick Examples: patient login authentication, merge UHID records, appointment booking, password reset OTP

SEARCH

DEDUPLICATE

SUMMARIZE WITH AI

COPY ALL RESULTS

Dedup Threshold: 0.85, Summary Type: Concise, Show Duplicates

SEARCH RESULTS

DEDUPLICATED

AI SUMMARY

Deduplication Statistics

Original Count: 20, After Deduplication: 18, Duplicates Removed: 2, Reduction: 10.0%

Unique Results (18)

TC_027, Unique, Score: 0.8175, Merge UHID

TC_028, Unique, Score: 0.9054, Demerge UHID

TC_047, Unique, Score: 0.8890, Converted to UHID

TC_006, Unique, Score: 0.8860, OLD to New UHID conversion

Elements

Console

Sources

Network

Performance

Memory

Application

Filter

Fetch/XHR

Doc

CSS

JS

Font

Img

Media

Manifest

Socket

Wasm

Other

100,000 ms

200,000 ms

300,000 ms

400,000 ms

500,000 ms

600,000 ms

Name

hybrid

distinct

rerank

search

deduplicate

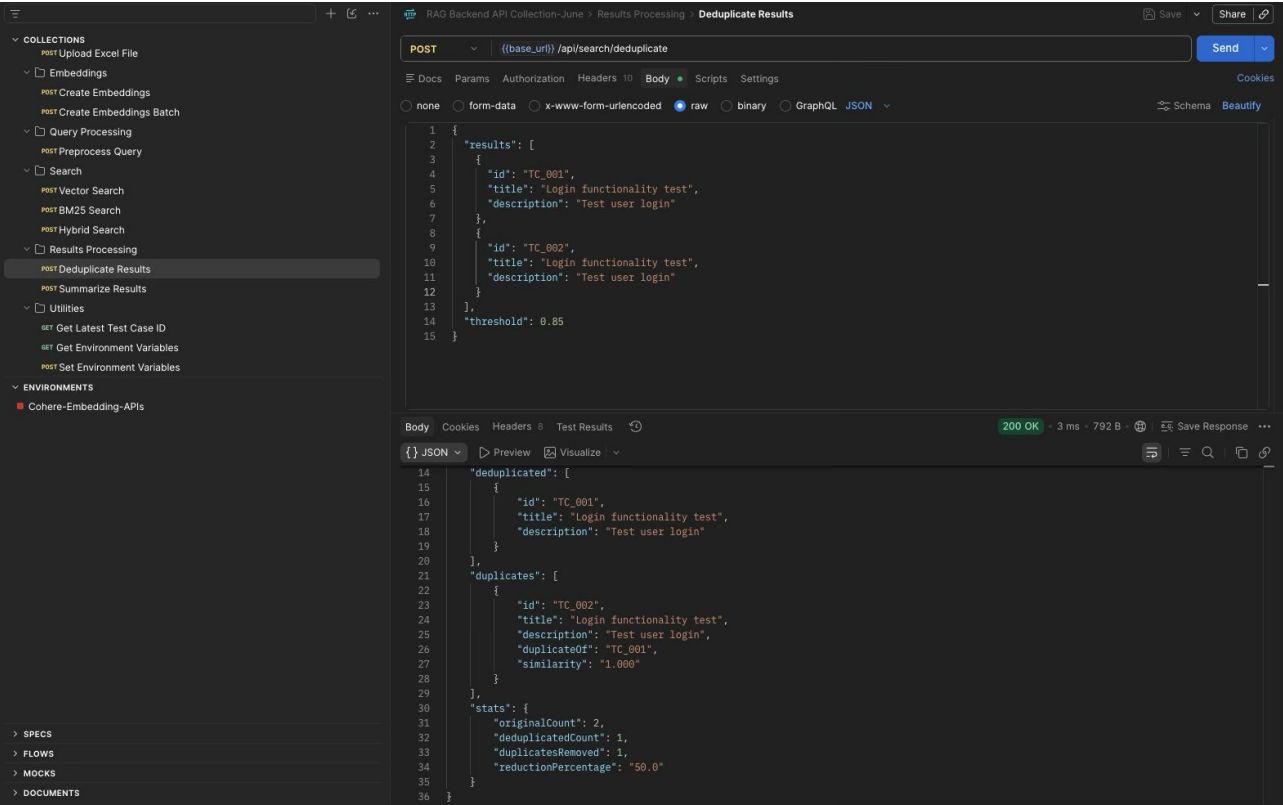
Request Payload

View source

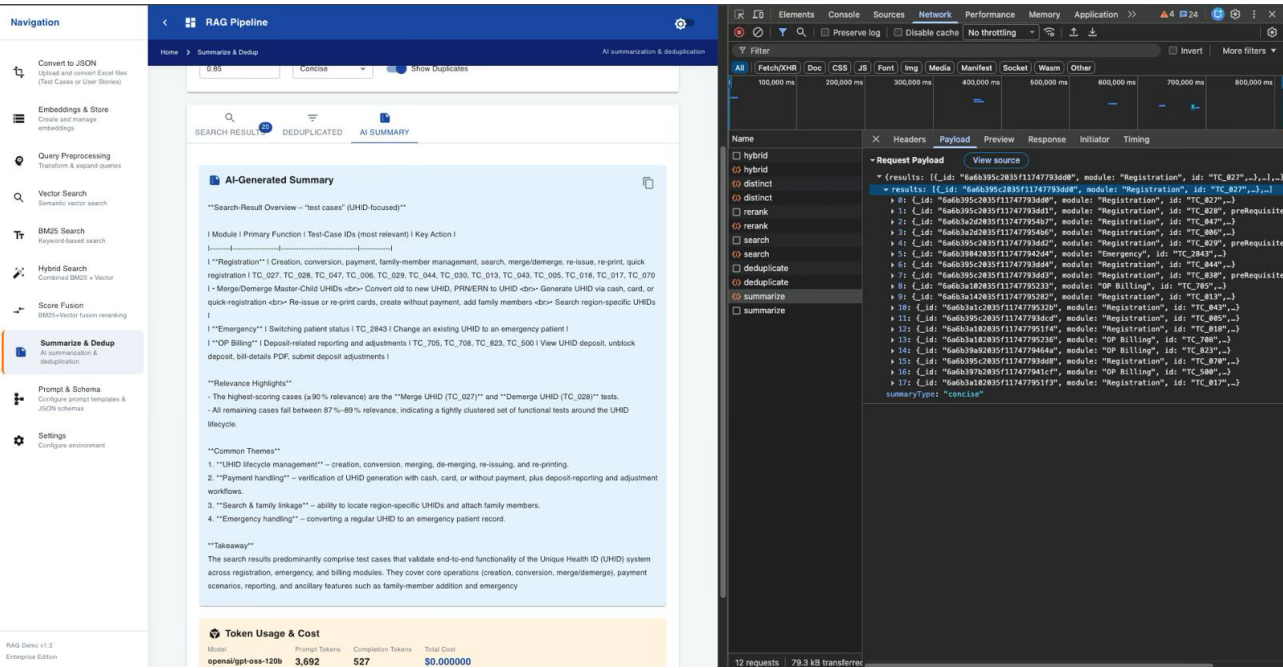
(results: [{_id: "6a6b395c2835f11747793d9d", module: "Registration", id: "TC_827",...}], ...)

threshold: 0.85

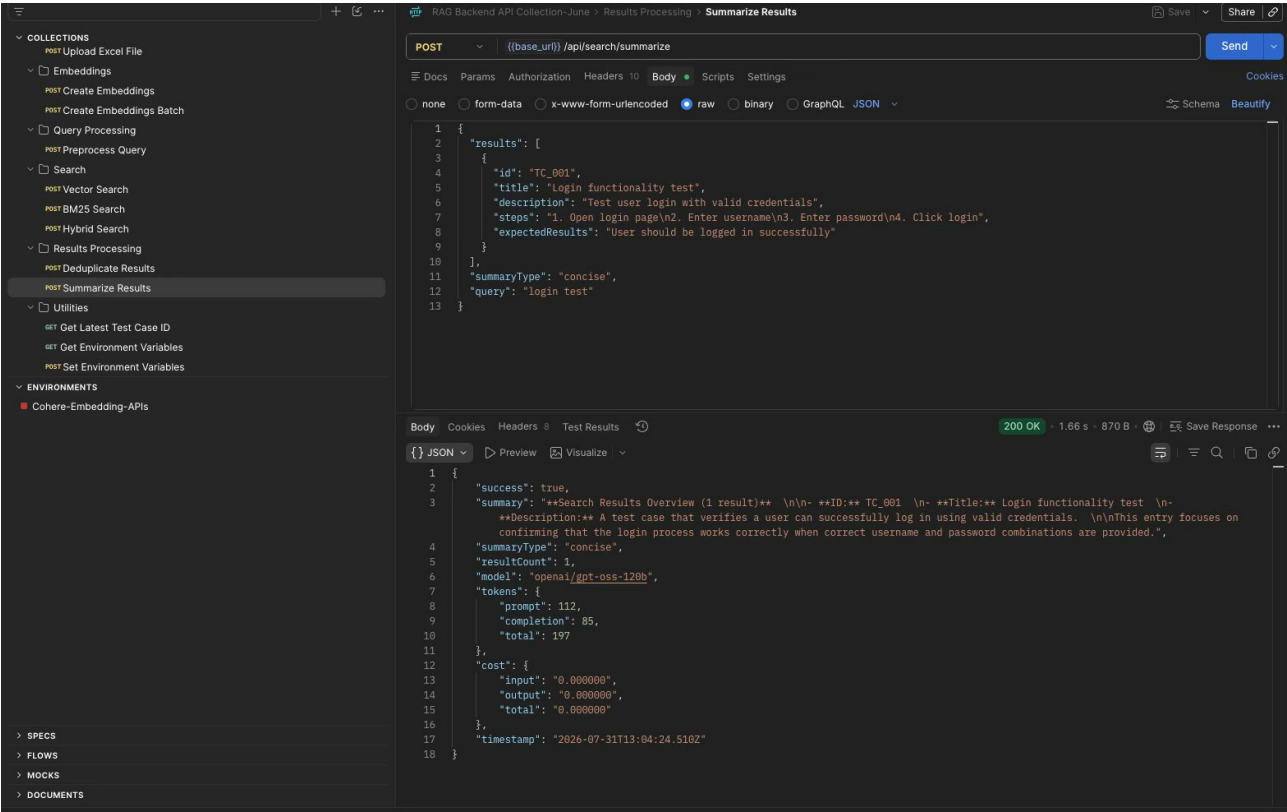
Deduplication using Postman API



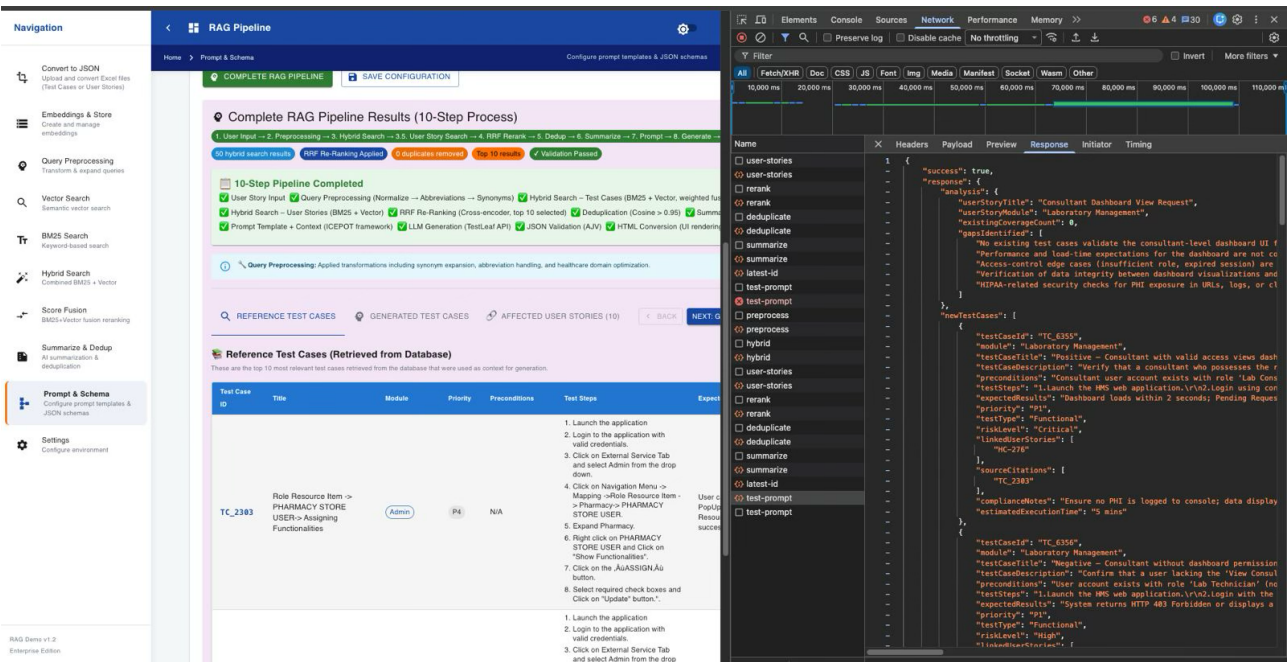
Step 7: Summarise with AI



Summarisation using postman API



Step 8: Complete RAG pipeline



ASSIGNMENT 2: EXPLORATION OF SARVAM AI APIs THROUGH POSTMAN

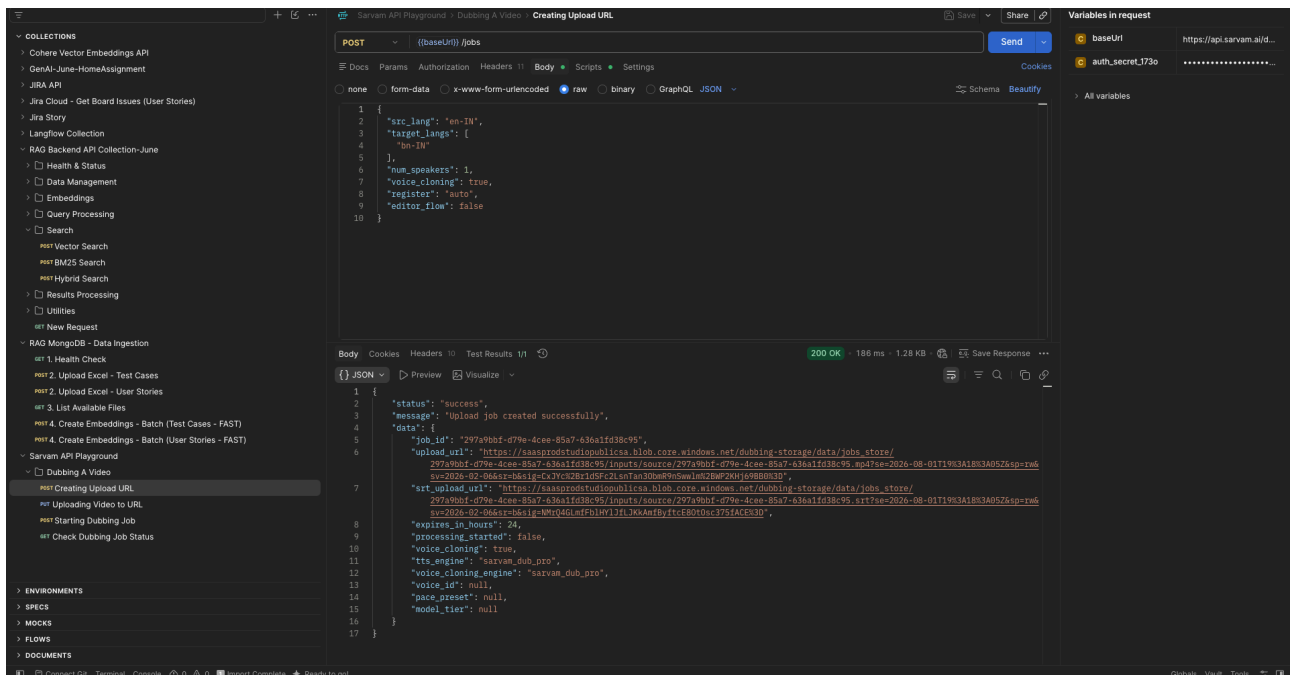
This API takes in a video and dubs it into a different language

Step 1: Create an upload URL link using the <https://api.sarvam.ai/dubbing/jobs> endpoint.
The API takes in the body structure below, where `src_lang` takes the source language, and `target_langs` takes the dubbed language details

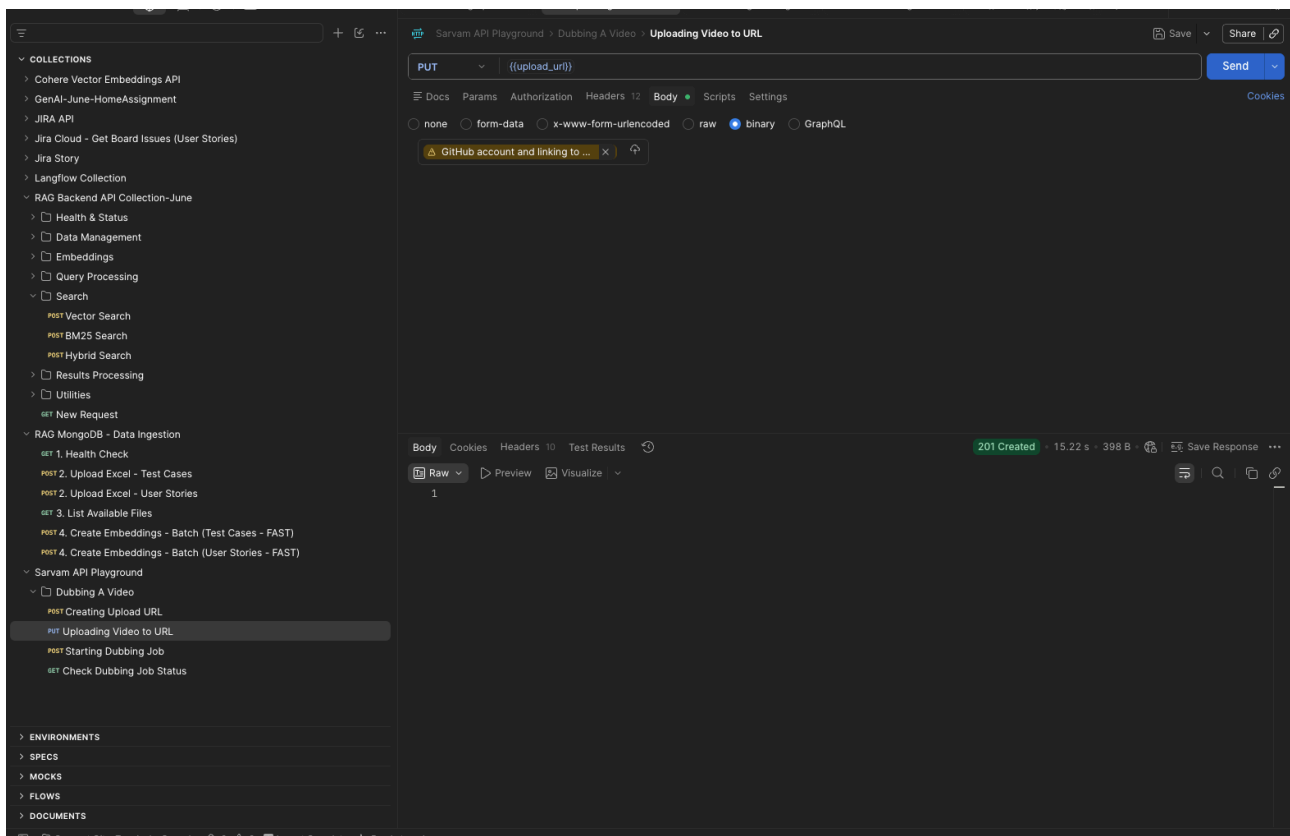
```
{
  "src_lang": "en-IN",
  "target_langs": [
    "bn-IN"
  ],
  "num_speakers": 1,
  "voice_cloning": true,
  "register": "auto",
  "editor_flow": false
}
```

The response for this API is an upload url link where the video needs to be uploaded

```
{
  "status": "success",
  "message": "Live status retrieved successfully",
  "data": {
    "job_id": "297a9bbf-d79e-4cee-85a7-636a1fd38c95",
    "job_name": null,
    "status": "completed",
    "current_step": "completed",
    "current_step_label": "Completed",
    "progress": 100,
    "export": {
      "target_language": "bn-IN",
      "status": "completed",
      "dubbed_video_url": "https://saas-studio-media-bkdpcygfbgaraxgr.a01.azurefd.net/dubbing-storage/data/jobs_store/297a9bbf-d79e-4cee-85a7-636a1fd38c95/exports/0406b5a8-6dc0-43af-ad4c-b4f4b0738c17/Bengali.mp4?se=2026-08-01T19%3A28%3A25Z&sp=r&sv=2026-02-06&sr=b&sig=dYnYHR4DaBzpAYcZe5bPqiEhluUCVgZf5EUGFvWNQVs%3D",
      "original_video_url": "https://saas-studio-media-bkdpcygfbgaraxgr.a01.azurefd.net/dubbing-storage/data/jobs_store/297a9bbf-d79e-4cee-85a7-636a1fd38c95/inputs/source/297a9bbf-d79e-4cee-85a7-636a1fd38c95.mp4?se=2026-08-01T19%3A28%3A25Z&sp=r&sv=2026-02-06&sr=b&sig=v%2BprSr1qFckTqPAJFhEgTTjaaOcokWvsGOe6tqB5p8Y%3D",
      "duration": null,
      "file_size": null
    },
    "exports": null,
    "error_message": null
  }
}
```

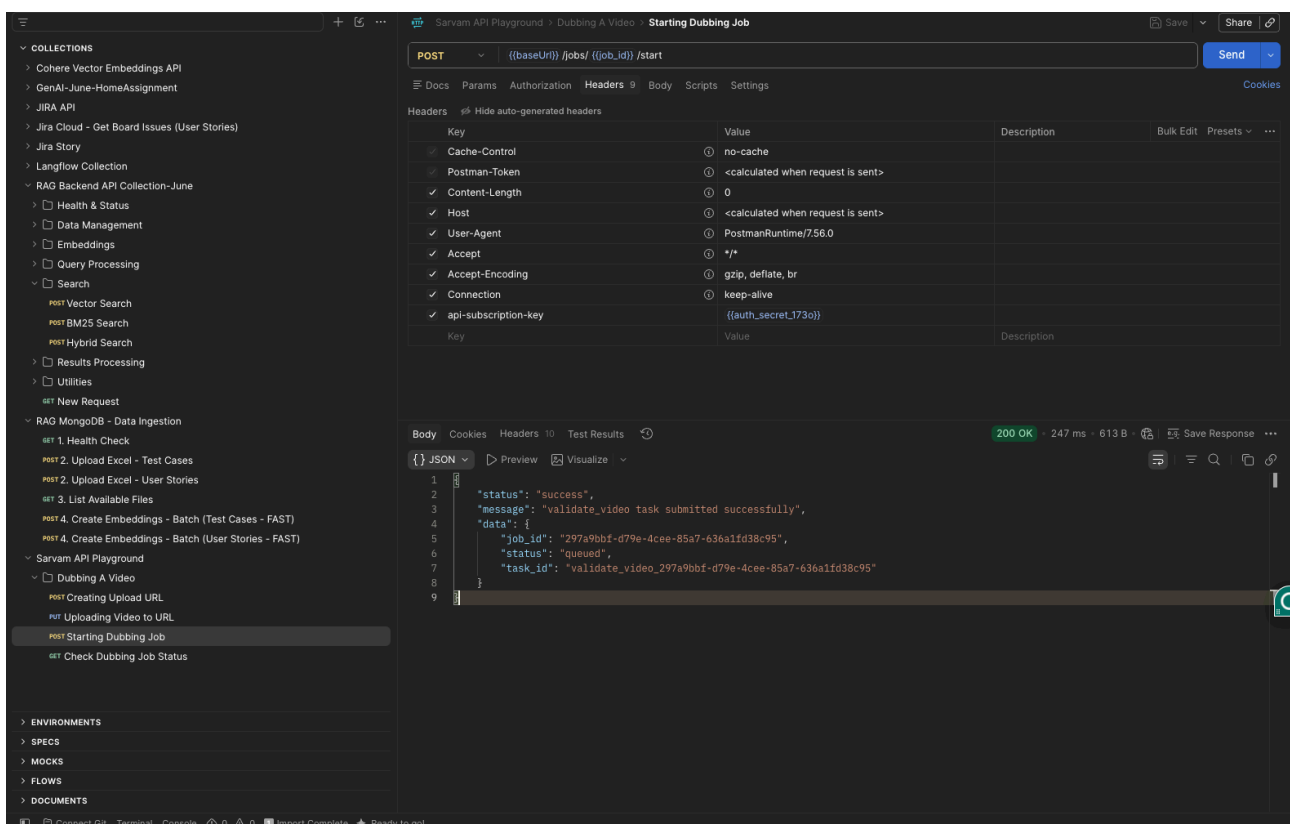
Step 2: Upload the video to the upload URL created in Step 1. The API uses the upload URL as the endpoint and the PUT method to upload the source video to that URL. The source video needs to be added as a binary in the body section of the API. The response of this API is HTTP code 201



Step 3: Start the dubbing job. The API endpoint is https://api.sarvam.ai/dubbing/jobs/:job_id/start. The job_id is generated as a response to Step 1. The API uses the POST method and doesn't include a payload.

The response for this API is a status message containing details of the initiated dubbing job

```
{
  "status": "success",
  "message": "validate_video task submitted successfully",
  "data": {
    "job_id": "297a9bbf-d79e-4cee-85a7-636a1fd38c95",
    "status": "queued",
    "task_id": "validate_video_297a9bbf-d79e-4cee-85a7-636a1fd38c95"
  }
}
```



Step 4: Tracking Job Status. The API endpoint is https://api.sarvam.ai/dubbing/jobs/:job_id/export-status. The job_id is generated as a response to Step 1. The API uses the GET method to check the execution status of the dubbing Job.

The response for this API is a status message containing final details of the initiated dubbing job. This API needs to be polled every 10-15 seconds until the status changes to **Success**.

The response contains a download link from where the original video and the dubbed video can be downloaded

```
{
  "status": "success",
  "message": "Live status retrieved successfully",
  "data": {
```

```

"job_id": "297a9bbf-d79e-4cee-85a7-636a1fd38c95",
"job_name": null,
"status": "completed",
"current_step": "completed",
"current_step_label": "Completed",
"progress": 100,
"export": {
  "target_language": "bn-IN",
  "status": "completed",
  "dubbed_video_url": "https://saas-studio-media-bkdpcygfbgaraxgr.a01.azurefd.net/dubbing-storage/data/jobs_store/297a9bbf-d79e-4cee-85a7-636a1fd38c95/exports/0406b5a8-6dc0-43af-ad4c-b4f4b0738c17/Bengali.mp4?se=2026-08-01T19%3A28%3A25Z&sp=r&sv=2026-02-06&sr=b&sig=dYnYHR4DaBzpAYcZe5bPqiEhluUCVgZf5EUGFvWNQVs%3D",
  "original_video_url": "https://saas-studio-media-bkdpcygfbgaraxgr.a01.azurefd.net/dubbing-storage/data/jobs_store/297a9bbf-d79e-4cee-85a7-636a1fd38c95/inputs/source/297a9bbf-d79e-4cee-85a7-636a1fd38c95.mp4?se=2026-08-01T19%3A28%3A25Z&sp=r&sv=2026-02-06&sr=b&sig=v%2BprSr1qFckTqPAJFhEgTTjaaOcoKwvsGOe6tqB5p8Y%3D",
  "duration": null,
  "file_size": null
},
"exports": null,
"error_message": null
}
}

```

The screenshot displays the Postman API Playground interface. The left sidebar shows a collection of API collections, with 'Sarvam API Playground' expanded. The main area shows a GET request to the endpoint `/{baseUri}/jobs/{job_id}/live-status`. The request headers are configured with various keys and values, including `Cache-Control`, `Postman-Token`, `Host`, `User-Agent`, `Accept`, `Accept-Encoding`, `Connection`, and `api-subscription-key`. The response is a 200 OK status with a 353 ms response time and 1.35 KB of data. The response body is shown in JSON format, containing the job details as defined in the JSON snippet above.

Key	Value	Description
Cache-Control	no-cache	
Postman-Token	<calculated when request is sent>	
Host	<calculated when request is sent>	
User-Agent	PostmanRuntime/7.56.0	
Accept	/*	
Accept-Encoding	gzip, deflate, br	
Connection	keep-alive	
api-subscription-key	{auth_secret:1730}	

```

{
  "data": {
    "job_id": "297a9bbf-d79e-4cee-85a7-636a1fd38c95",
    "job_name": null,
    "status": "completed",
    "current_step": "completed",
    "current_step_label": "Completed",
    "progress": 100,
    "export": {
      "target_language": "bn-IN",
      "status": "completed",
      "dubbed_video_url": "https://saas-studio-media-bkdpcygfbgaraxgr.a01.azurefd.net/dubbing-storage/data/jobs_store/297a9bbf-d79e-4cee-85a7-636a1fd38c95/exports/0406b5a8-6dc0-43af-ad4c-b4f4b0738c17/Bengali.mp4?se=2026-08-01T19%3A28%3A25Z&sp=r&sv=2026-02-06&sr=b&sig=dYnYHR4DaBzpAYcZe5bPqiEhluUCVgZf5EUGFvWNQVs%3D",
      "original_video_url": "https://saas-studio-media-bkdpcygfbgaraxgr.a01.azurefd.net/dubbing-storage/data/jobs_store/297a9bbf-d79e-4cee-85a7-636a1fd38c95/inputs/source/297a9bbf-d79e-4cee-85a7-636a1fd38c95.mp4?se=2026-08-01T19%3A28%3A25Z&sp=r&sv=2026-02-06&sr=b&sig=v%2BprSr1qFckTqPAJFhEgTTjaaOcoKwvsGOe6tqB5p8Y%3D",
      "duration": null,
      "file_size": null
    },
    "exports": null,
    "error_message": null
  }
}

```